Backend Development Roadmap

Node.js · E-Commerce Project · Beginner to Pro

Phase 1 — Node.js & HTTP fundamentals

2–3 weeks · Express, REST APIs, routing, middleware, JSON

■ Product listing API

GET /products · POST /products with Express · no DB yet

▼

Phase 2 — Databases

3 weeks · PostgreSQL, Prisma ORM, MongoDB basics, SQL queries

■ Persist products + users in DB

CRUD operations · relations · schema design

▼

Phase 3 — Auth & security

2–3 weeks · JWT, bcrypt, sessions, role-based access, input validation

■ Login + protected cart routes

Admin vs buyer roles · JWT middleware

▼

Phase 4 — Advanced backend patterns

3–4 weeks · File uploads, Stripe payments, emails, background jobs

■ Checkout, payments, order emails

Stripe integration · Nodemailer invoices

▼

Phase 5 — Production & DevOps basics

2–3 weeks · Docker, CI/CD, env config, logging, error handling

■ Deploy app to Railway / Render

Dockerize · logging · set env secrets

▼

Phase 6 — Pro-level skills

Ongoing · Redis caching, testing, rate limiting, API versioning, system design

■ Full production-ready e-commerce backend

Products · Auth · Cart · Orders · Payments · Deployed

Phase 1 — Start Today: Node.js & HTTP Fundamentals

You already know frontend JS, so Node.js will feel familiar. The big shift is thinking server-side — handling requests, sending responses, and structuring code that runs on a machine, not a browser.

Week 1 — Setup + your first server

Install Node.js — Download from nodejs.org — choose the LTS version.

Learn npm basics — npm init, npm install, package.json, node_modules.

Write a basic HTTP server — Use Node's built-in http module. Understand req and res.

Install Express — npm install express — and rewrite your server in 10 lines.

Learn routing — app.get(), app.post(), app.put(), app.delete()

Understand middleware — app.use(), body-parser, how requests flow through layers.

Week 2 — REST API design

REST principles — Resources, HTTP verbs (GET/POST/PUT/DELETE), status codes (200, 201, 404, 500).

JSON responses — Always respond with JSON. Use res.json({}) not res.send().

Route params — /products/:id — dynamic URLs. Query strings: /products?category=shoes.

Error handling — Centralized error middleware. Never let errors crash the server.

Postman / Thunder Client — Test your API without a frontend. Essential tool.

Week 3 — Build the project: Product Listing API

Build these endpoints. Store data in a JS array (no database yet — that's Phase 2):

Method	Route	What it does
GET	/products	Return all products
GET	/products/:id	Return one product by ID
POST	/products	Add a new product
PUT	/products/:id	Update a product
DELETE	/products/:id	Delete a product
GET	/products/search	Filter by category or price

Starter project structure:

```
my-ecommerce-backend/
■■■ index.js ← entry point, starts the server
■■■ package.json
■■■ routes/
■ ■■■ products.js ← all /products routes
■■■ controllers/
```

```
■ ■■■ productController.js ← logic for each route
■■■ data/
■■■ products.js ← your in-memory array for now
```

Resources to use:

- Node.js docs — nodejs.org/en/docs
- Express.js docs — expressjs.com
- The Odin Project (free) — theodinproject.com/paths/full-stack-javascript
- YouTube: Traversy Media 'Node.js Crash Course'
- REST API design guide — restfulapi.net

Building the Full E-Commerce Website

Your backend knowledge + your existing frontend skills = a complete full-stack app. Here's how they connect:

Layer	Technology	What you already know
Frontend UI	React or vanilla JS	You know this — fetch() calls your own API
API layer	Express (Node.js)	Phase 1 — you'll build this
Database	PostgreSQL + Prisma	Phase 2 — products, users, orders
Auth	JWT tokens	Phase 3 — login, sessions
Payments	Stripe	Phase 4 — checkout flow
Hosting	Railway / Render	Phase 5 — live on the internet

The rule: build backend first, connect frontend after each phase.

Don't try to build both at once. After Phase 1, spend a day connecting your React frontend to your product API using fetch(). After Phase 2, show real DB data. Each phase adds a layer to both the backend and the website simultaneously.

Final website pages to build:

Home page — Featured products, categories — calls GET /products

Product listing — Search, filter, sort — calls GET /products?category=...

Product detail — Single product page — calls GET /products/:id

Cart — Add/remove items — stored in localStorage until Phase 3

Login / Signup — Auth forms — calls POST /auth/login (Phase 3)

Checkout — Stripe payment form (Phase 4)

Order history — Past orders for logged-in user (Phase 4)

Admin dashboard — Manage products, view orders (Phase 3 — role-based)

One project. Six phases. Start today with Phase 1 — your first Express server.